

Trabajo practico final

Diseño de interfaz gráfica y control de matriz Led

Alumno: Dinter, Franco

E-mail: francopiasensa@gmail.com

Fecha de entrega: 12/08/2025

Cátedra: Diseño digital con microcontroladores

1. Objetivo del Proyecto

El objetivo principal de este proyecto fue desarrollar un sistema que permita configurar y controlar una pantalla matricial de LEDs de 8x32 (compuesta por 4 matrices de 8x8) mediante un microcontrolador STM32 y el circuito integrado MAX7219.

Además, se buscó implementar una interfaz gráfica táctil utilizando *TouchGFX Designer*, de forma que el usuario pueda interactuar directamente con el hardware, configurar parámetros y visualizar en tiempo real el estado de la matriz. Este trabajo también tuvo como fin familiarizarse con las herramientas de desarrollo de STM32, incluyendo la configuración de pines, frecuencias, periféricos y la integración de librerías personalizadas.

2. Hardware Utilizado

- **Microcontrolador:** STM32 Disco
- **Protoboard** para montaje de prueba
- **Regulador de voltaje:** LM7805
- **Módulo Step-Up / Step-Down** para control de tensión de alimentación
- **4 matrices LED 8x8**
- **Circuito integrado:** MAX7219 (controlador de matriz)
- **Capacitores** para filtrado de alimentación

3. Software Utilizado

- **TouchGFX Designer:** para el diseño de la interfaz gráfica y generación del código asociado.
- **STM32CubeIDE:** entorno principal de programación y compilación del firmware.
- **STM32CubeMX:** para la configuración de periféricos, pines y generación de código de inicialización.

4. Problemática que Resuelve

El proyecto resuelve la necesidad de controlar cuatro matrices de LEDs 8x32 de manera interactiva y configurable, permitiendo:

- Seleccionar patrones de encendido directamente desde una **pantalla táctil**.
- Reflejar de forma inmediata los cambios realizados en la interfaz gráfica sobre la matriz física.
- Visualizar gráficamente el progreso de la ejecución de un patrón.
- Integrar la lógica de control del chip MAX7219 con un sistema de interfaz amigable para el usuario, usando el protocolo SPI para la comunicación.

- La finalidad es la integración a posteriori con un proyecto más ambicioso, generando diferentes patrones lumínicos para detectar movimientos oculares y mostrar los resultados en pantalla.

5. Tecnologías Implementadas

- **Hardware:**
 - Microcontrolador STM32 con pines SPI configurado para comunicación con el MAX7219.
 - Pantalla táctil integrada en la placa de desarrollo.
 - Reguladores y etapas de alimentación.
- **Software y Librerías:**
 - Código autogenerado por TouchGFX para la interfaz.
 - Código de inicialización del SPI generado por STM32CubeMX.
 - Librería personalizada para control del MAX7219, estructurada en funciones modulares que permiten configurar registros y enviar datos a la matriz.

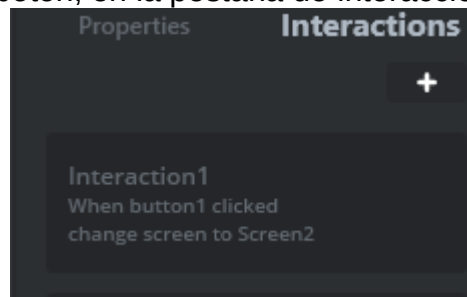
6. Configuración del Sistema

1. Diseño de la Interfaz:

- En *TouchGFX Designer* se diseñaron varias pantallas con botones, sliders, barras de progreso y textos dinámicos (*wildcards*).
- Cada elemento interactivo se configuró con *callbacks* que, al ser activados, ejecutan funciones específicas del firmware.

2. Configuración de Botones en TouchGFX:

- Una vez creado el botón, en la pestaña de interacciones se crea una



seleccionando el +:

- En el menu de interacciones se selecciona el boton que realizara la accion y con que se la llamara

- A lo largo del proyecto se usaron dos configuraciones. Un para moverse entre la interfaz, donde se selecciona a que pantalla se desea ir:

Trigger
Button is clicked

Choose clicked source
button1

Action
Change screen

Choose screen
Screen2

Transition
Slide

- La segunda es ejecutar un código en C++. Esto cuando se selecciona, al generar el código, al momento de seleccionar el botón ejecuta de forma automática el código que le pasemos, además se debe incluir donde se encuentra la función que se desea ejecutar:

Interaction2

Trigger
Button is clicked

Choose clicked source
inc

Action
Execute C++ code

Code
presenter->updateBrilloValue (1...

Includes
#include <gui/screen1_screen/S...

☐ Can trigger another interaction

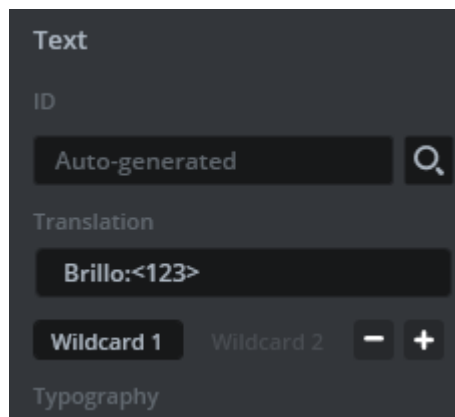
Interaction Name
Interaction2

```
#include <gui/screen1_screen/Screen1Presenter.hpp>
```

- Luego de generar el código, abrir el archivo correspondiente al **Presenter** de la pantalla (por ejemplo, **Screen1Presenter.cpp**).
- Declarar la función indicada en el paso anterior.
- Dentro de esta función, escribir la lógica que se ejecutará al presionar el botón (por ejemplo, enviar un comando al MAX7219 vía SPI).

3. Configuración Wildcards:

- En TouchGFX Designer, arrastrar un **TextArea** a la pantalla.
- En el panel **Text**, seleccionar la opción **Wildcard**.



- En el apéndice se mostrara el código para actualizar el valor de la wildcard.

4. Configuración de Pines y SPI:

Con *STM32CubeMX* se seleccionaron los pines necesarios para la comunicación SPI con el MAX7219:

- **CS** (*Chip Select*)
- **MOSI** (*Master Out Slave In*)
- **CLK** (*Clock*)
- El código generado inicializa el bus SPI y define el handler correspondiente, utilizado luego por la librería del MAX7219.

5. Librería del MAX7219:

- Se desarrolló una librería propia que encapsula funciones para:

- Configuración de registros (intensidad, modo de operación, decodificación, apagado/encendido).
- Envío de datos a la matriz vía SPI.
- Control individual o por bloques de LEDs.

7. Funcionamiento del Código

El flujo de funcionamiento se organiza en pantallas:

Pantalla 1 – Ajuste de Intensidad

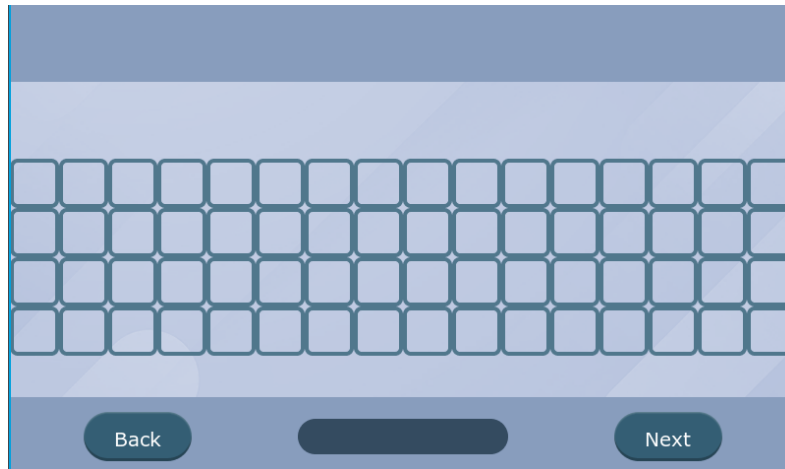


- Se permite regular el brillo de la matriz mediante botones de incremento y decremento.
- Para modificar el brillo se envía por SPI, dirección del registro que almacena la intensidad de los LED's del chip MAX7219 y el valor deseado de intensidad
- El valor se actualiza en pantalla usando wildcards y se refleja de inmediato en el registro correspondiente del MAX7219.
- La interacción se gestiona con funciones asociadas a los eventos táctiles de los botones. Como fue explicado en la configuración del sistema, configuración de botones

La función que se utilizó en la configuración del botón:

```
presenter->updateBrilloValue(1); // incrementar
presenter->updateBrilloValue(0); // decremento
```

Pantalla 2 – Selección de LEDs:

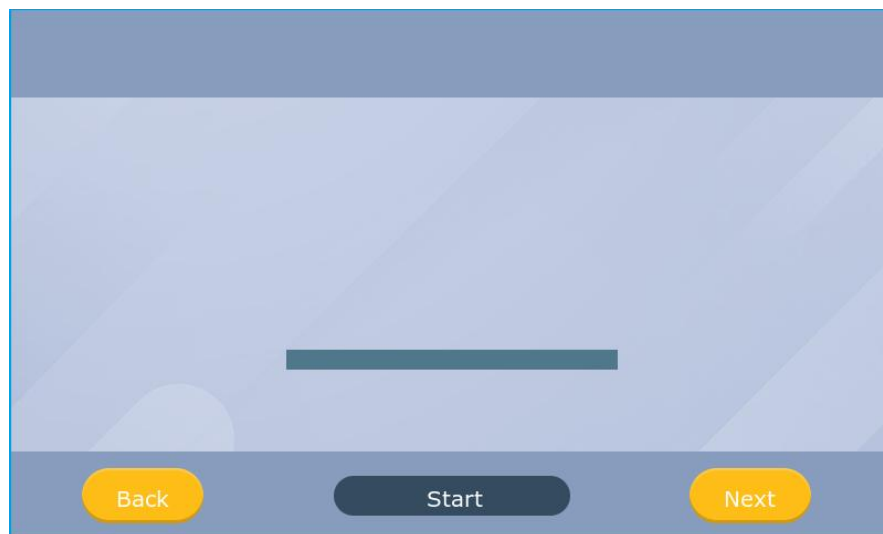


- La pantalla muestra una cuadrícula donde cada botón representa una zona de 2×2 LEDs.
- Al presionar un botón, se envían coordenadas (matriz, fila, columna) para encender de inmediato los LEDs correspondientes. Por ejemplo:

```
presenter->selectMatrizLed(0, 4 , 0);
```

- La selección se refleja simultáneamente en la matriz física.

Pantalla 3 – Ejecución de Patrones:



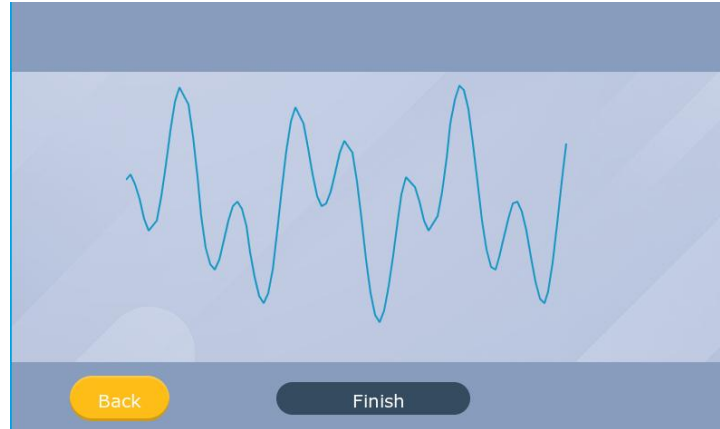
- Un botón *Start* inicia el patrón predefinido en la matriz.

```
presenter->Start_Pattern();
```

- Se actualiza una **barra de progreso** que indica gráficamente el avance del proceso.

- Debido a que la interfaz funciona bajo **FreeRTOS**, no se utilizan HAL_delays directos, en su lugar, se crea una tarea específica que gestiona la temporización. De esta forma el proceso actualización de la pantalla, es no bloqueante, es decir no interrumpe la ejecución del hilo principal que controla la pantalla.

Pantalla 4 – Gráfico



- Se muestra un gráfico cartesiano con puntos generados a partir de una señal senoidal, a modo de demostración del control gráfico.
- Aunque no se presentan datos reales, sirve como ejemplo de visualización.

8. Posibles Mejorar

- Pasar el *handler* de SPI como puntero a la función de inicialización para mejorar la modularidad.
- Ampliar la librería del MAX7219 para soportar animaciones y patrones más complejos.
- Implementar almacenamiento de configuraciones y patrones en una tarjeta SD, de tal forma que el usuario pueda .

9. Conclusión

El proyecto logró integrar de manera exitosa el control de una matriz de LEDs 8x32 mediante el circuito integrado MAX7219 y un microcontrolador STM32, complementado con una interfaz gráfica táctil desarrollada en *TouchGFX Designer*.

Se implementó una comunicación eficiente a través del protocolo SPI, accediendo y configurando registros del MAX7219 para administrar la intensidad, el encendido y el patrón de los LEDs en tiempo real.

La interfaz gráfica permitió al usuario interactuar de forma intuitiva con el hardware, seleccionando patrones, ajustando parámetros y visualizando el progreso de ejecución, todo con respuesta inmediata sobre la matriz. El uso de STM32CubeMX facilitó la configuración de pines y periféricos, mientras que la librería personalizada del MAX7219 garantizó un código modular y escalable para futuras ampliaciones. Durante el desarrollo se adquirieron conocimientos clave en:

- Diseño y configuración de interfaces gráficas con TouchGFX.
- Configuración de periféricos de comunicación SPI en STM32.
- Integración de hardware y software bajo un entorno de **FreeRTOS**.
- Estructuración de código mediante librerías personalizadas.

En síntesis, el proyecto no solo cumplió con el objetivo de controlar la matriz y crear una interfaz funcional, sino que también proporcionó una sólida experiencia en herramientas de desarrollo, diseño de interfaces y manejo de periféricos, dejando una base robusta para implementaciones más complejas y mejoras futuras.

Apéndice:

En esta sección se resaltarán los códigos que se creen más importantes, a su vez todo el código se encuentra subido en GitHub que será agregado al final del informe.

Creación de tarea para actualizar pantalla de carga de forma no bloqueante:

```
WaitingScreenPresenter::WaitingScreenPresenter(WaitingScreenView& v)
    : view(v)
{
    ledTaskHandle = NULL; // inicializo el handle que se encarga de la
    tarea
    progressQueue = xQueueCreate(5, sizeof(uint8_t));
}

void WaitingScreenPresenter::activate()
{
}

void WaitingScreenPresenter::deactivate()
{
    // si ya hay una tarea la elimino
    if(ledTaskHandle != NULL)
    {
        vTaskDelete(ledTaskHandle);
        ledTaskHandle = NULL;
    }
}

void WaitingScreenPresenter::Start_Pattern()
```

```

{
    // Creo la tarea que se encarga de los leds
    xTaskCreate(
        ledDisplayTask,        // Función de la tarea
        "LEDTask",             // Nombre de la tarea
        512,                   // Tamaño del stack (ajustar según necesites)
        this,                  // Parámetro (pasar 'this' para acceder al
presenter)
        2,                     // Prioridad (2 = alta prioridad)
        &ledTaskHandle         // Handle de la tarea
    );
}
void WaitingScreenPresenter::ledDisplayTask(void* pvParameters)
{
    // Convertir el parámetro de vuelta al presenter
    WaitingScreenPresenter* presenter =
static_cast<WaitingScreenPresenter*>(pvParameters);

    // Aquí va tu función original, pero modificada
    for(uint8_t i = 0; i < in_data_count; i++)
    {
        MAX7219_ShowSeleccctions(i);
        vTaskDelay(pdMS_TO_TICKS(TIME_DELAY_SHOW_LED));

        uint8_t progress = ((i + 1) * 100) / in_data_count;

        xQueueSend(presenter->progressQueue, &progress, 0);
    }

    MAX7219_ClearAll();

    uint8_t finalProgress = 100;
    xQueueSend(presenter->progressQueue, &finalProgress, 0);
    // Elimino la tarea ya termine
    presenter->ledTaskHandle = NULL;
    vTaskDelete(NULL);
}
void WaitingScreenPresenter::checkProgressUpdate()
{
    uint8_t progress;

    // Verificar si hay datos en la cola (no bloquear)
    if(xQueueReceive(progressQueue, &progress, 0) == pdTRUE)
    {
        view.ProgressBar(progress);
    }
}

```

Generación de función sinodal y graficada en pantalla:

```

GraficView::GraficView()

```

```

{
}

void GraficView::setupScreen()
{
    GraficViewBase::setupScreen();
    graph1.clear(); // limpio la pantalla
    const float amplitud = 50.0f;
    const float offset = 50.0f;
    const float frecuencia = 2.0f; // ciclos en el rango X
    const int numPuntos = 100;
    for (int x = 0; x <= numPuntos; x++)
    {
        float angulo = 2.0f * M_PI * frecuencia * (x /
(float)numPuntos);
        float valorY = offset + amplitud * sin(angulo);
        graph1.addDataPoint((int16_t)x, (int16_t)valorY);
    }
}

void GraficView::tearDownScreen()
{
    GraficViewBase::tearDownScreen();
}

```

Función que actualiza la wildcard de brillo:

```

GraficView::GraficView()
{
}

void GraficView::setupScreen()
{
    GraficViewBase::setupScreen();
    graph1.clear(); // limpio la pantalla
    const float amplitud = 50.0f;
    const float offset = 50.0f;
    const float frecuencia = 2.0f; // ciclos en el rango X
    const int numPuntos = 100;
    for (int x = 0; x <= numPuntos; x++)
    {
        float angulo = 2.0f * M_PI * frecuencia * (x /
(float)numPuntos);
        float valorY = offset + amplitud * sin(angulo);
        graph1.addDataPoint((int16_t)x, (int16_t)valorY);
    }
}

```

```
void GraficView::tearDownScreen()  
{  
    GraficViewBase::tearDownScreen();  
}
```

GitHub para acceder al código completo:
https://github.com/DinterFran/TPFinal_Ojos